Continual Medical Image Classification on BMAD: Rehearsal, Regularization and Forgetting in a Domain-Incremental Stream

Peter Chika Ozo-ogueji
American University, Washington, DC

Faculty mentor: Prof. Roberto Corizzo, Department of Computer Science, American University

Preliminary technical report • November 2026

Abstract

Medical imaging models must adapt to new anatomies, modalities and sites without retraining on all past data, which is often impossible to retain for privacy reasons; learning tasks in sequence, however, causes catastrophic forgetting. We build a supervised, domain-incremental benchmark from BMAD, following the task stream of Barusco et al. [4]: brain MRI, liver CT, two retinal OCT datasets, chest X-ray and histopathology, each posed as normal-versus-abnormal classification. Because BMAD provides labeled abnormal images only outside its training folders, we re-split each category into training, validation and test sets grouped by patient, volume or slide block, verified by an independent leakage audit. We compare six continual-learning methods in the Mammoth framework under a protocol that tunes all hyperparameters on validation only and reports test results over three seeds with 95% confidence intervals. DER++ and Experience Replay reach a final average AUROC of 0.915 ± 0.034 and 0.914 ± 0.036, within about 0.05 of a tuned offline joint-training reference (0.966 ± 0.006), whereas fine-tuning ends at 0.612 ± 0.033 and online EWC, SI and A-GEM at 0.65–0.72. In a paired same-seed comparison, DER++ improves on fine-tuning by 0.30 AUROC (95% CI $[0.24, 0.37]$). Forgetting can invert decision rules (AUROC 0.40 on histopathology under fine-tuning), task order shifts fine-tuning by 0.08 AUROC but replay by at most 0.02, and online EWC is numerically unstable at strong regularization. We document the BMAD data issues found while building the benchmark, including filename conventions that silently break patient grouping.

1 Introduction

A clinical imaging model rarely sees all of its data at once. Hospitals add scanners, new sites join, and new anatomies or modalities come into scope. Retraining from scratch on every image ever collected is often impossible, because patient data may not be retained or shared across institutions. Continual learning (CL) studies how a model can learn from such a sequence of tasks while preserving what it learned earlier.

The central obstacle is *catastrophic forgetting* [19]. All tasks share one set of parameters, but each gradient step sees only the current task's loss, so nothing protects the features and decision boundaries that earlier tasks relied on. Medical imaging makes the problem harder: consecutive tasks can differ in modality, resolution, intensity statistics and class balance, and the data available per task are small.

Qazi et al. [20] survey CL in medical imaging and observe that evaluations are difficult to compare: studies use different datasets, task definitions and orders, and task order alone can change results substantially. Barusco et al. [4] proposed a concrete multi-modality stream built from the BMAD benchmark [3], but for continual *anomaly detection*, in which models are trained on normal images only. The supervised setting, in which a model learns from labeled normal and abnormal images, is to our knowledge not yet covered by an equivalent standardized stream.

This report builds that supervised benchmark and evaluates standard CL methods on it. Our contributions are:

1. **A supervised, domain-incremental BMAD stream:** six tasks in the order of Barusco et al. [4], each normal-versus-abnormal, with a documented re-split grouped by patient, volume or slide block.
2. **A leakage-aware evaluation protocol:** an independent split audit (group overlap and duplicate files), a shortcut audit, validation-only tuning of all hyperparameters including the joint-training reference, three seeds with 95% intervals, paired same-seed comparisons, and group-level bootstrap intervals; failed runs are reported rather than discarded.
3. **An empirical comparison of six methods** (fine-tuning, ER, DER++, A-GEM, online EWC, SI) against joint-training and per-category references, with per-category, task-order and failure analyses.
4. **A data audit of BMAD for supervised use,** identifying filename conventions, source mixtures and grouping limits that follow-up studies should account for.

The main result is a clear separation between method families: replay methods retain almost all prior knowledge, while regularization methods and A-GEM end close to fine-tuning.

2 Related Work

Continual learning scenarios. van de Ven and Tolias [24] distinguish task-, domain- and class-incremental learning by what the model must infer at test time. In domain-incremental learning the label space is fixed while the input distribution changes, which matches our setting: every task asks "normal or abnormal?" while the anatomy and modality change. Lopez-Paz and Ranzato [18] introduced the accuracy matrix and backward transfer used to quantify forgetting.

Method families. *Regularization* methods penalize changes to parameters judged important for earlier tasks: elastic weight consolidation (EWC) [14], its online variant [22] and synaptic intelligence (SI) [25]. *Rehearsal* methods keep a small memory of past examples: Experience Replay (ER) [9] mixes them into training batches; DER++ [7] additionally stores the model's past logits and matches them, a form of self-distillation; A-GEM [8] uses the memory only to project gradients that would increase the loss on past examples. Knowledge distillation without memory, as in LwF [16], forms a third family. We use the Mammoth framework [1, 6, 7], which implements these methods under a single training loop.

Continual learning in medical imaging. Qazi et al. [20] identify practical obstacles that motivate this work: the lack of standardized benchmarks, inconsistent task definitions, sensitivity to task order, and privacy limits on storing past patient data. The last point matters when interpreting rehearsal results, since rehearsal is the family whose memory may be restricted in clinical deployment.

BMAD and continual anomaly detection. BMAD [3] collects six medical datasets for anomaly detection, with normal-only training folders. Barusco et al. [4] present these categories as a sequence and evaluate PatchCoreCL, a continual adaptation of PatchCore [21], reporting low forgetting with per-task memory banks; these anomaly-detection methods are implemented in the development version of PyCLAD [17]. Our work keeps their stream and order but changes the task to supervised classification. It therefore tests a different family of methods on the same scenario and is not a reproduction of PatchCoreCL.

3 Approach

3.1 Problem formulation

A network f_θ observes tasks $t = 1, \ldots, T$ in sequence ($T = 6$). Task t provides a training set $\mathscr{D}_t$ of images x with labels $y \in \{0, 1\}$ (normal, abnormal). While training on task t, the learner has no access to $\mathscr{D}_1, \ldots, \mathscr{D}_{t-1}$ except through a small memory $\mathscr{M}$ where the method permits one. After each task, it is evaluated on the test sets of all tasks seen so far. Because the label space is shared and only the input distribution changes, the setting is domain-incremental [24]. The network outputs two logits; the predicted probability of abnormality is the softmax of the abnormal logit.

3.2 Scenario construction

The task order follows Barusco et al. [4]: Brain MRI → Liver CT → Retina (RESC) → Chest X-ray → Histopathology → Retina (OCT2017). BMAD stores labeled abnormal images only in its validation and test folders, so supervised training requires a re-split, which is original to this work. For each category we:

1. **pool** all labeled images: abnormal images from the validation and test folders and normal images from all folders, except where normal training images come from a different source dataset than the labeled ones (Section 5.5);
2. **group** images by their independence unit, recovered from the filename format verified against the downloaded data: the subject or volume prefix (Brain, Liver, RESC), the RSNA patient identifier (Chest), the patient number in OCT2017's `DISEASE-PATIENT-IMAGE` names, and contiguous blocks of 25 patch numbers for Camelyon16, whose patch files carry no slide identifier;
3. **split** whole groups into test (30%), then validation (15%), leaving the remainder for training. BMAD training images that carry no identifier (RESC, Camelyon16) may enter only our training split, and only to fill a shortfall: training normals are drawn preferentially from the same source population as the validation and test normals, while patient-, volume- or block-level groups remain disjoint across splits;
4. **balance and cap** each split to a 1:1 class ratio, with at most 2,000 training, 500 validation and 1,000 test images per category;
5. **audit** the result independently: no group and no byte-identical file may appear in two splits.

Every split is deterministic given a seed, and a manifest with a SHA-256 checksum records each file's split and group; the checksum is logged with every result.

3.3 Methods

All methods share one network and optimizer and differ only in their training objective. **Fine-tuning** minimizes the cross-entropy ℓ on the current task alone and serves as the lower bound. **Experience Replay** adds the loss on a mini-batch drawn from a reservoir-sampled memory:

$$\mathscr{L}_{\mathrm{ER}} = \ell\big(f_\theta(x), y\big) + \ell\big(f_\theta(x'), y'\big), \qquad (x', y') \sim \mathscr{M}. \tag{1}$$

DER++ also stores the logits z' produced when x' entered the memory and matches them, with (x', z') and (x'', y'') two independent memory samples:

$$\mathscr{L}_{\mathrm{DER{+}{+}}} = \ell\big(f_\theta(x), y\big) + \alpha \left\| f_\theta(x') - z' \right\|_2^2 + \beta\, \ell\big(f_\theta(x''), y''\big). \tag{2}$$

Table 1: Task stream and final splits. Counts are images per class (normal and abnormal are balanced 1:1). Groups are the independence units used for splitting.

Task	Category	Modality	Source dataset	Train	Val	Test	Groups	Grouping unit
1	Brain	MRI slice	BraTS 2021 [2]	1,000	250	500	1,241	subject
2	Liver	CT slice	LiTS [5]	218	250	245	10	CT volume
3	Retina (RESC)	OCT	RESC [12]	398	174	262	15	OCT volume
4	Chest	X-ray	RSNA Pneumonia [23]	1,000	250	500	26,684	patient
5	Histopathology	H&E patch	Camelyon16 [10]	594	175	338	95	patch block*
6	Retina (OCT2017)	OCT	OCT2017 [13]	412	113	225	3,736	patient

*Camelyon16 patch files carry no slide identifier; blocks of 25 consecutive patch numbers approximate slides.

A-GEM computes the gradient g on the current batch and a reference gradient g_{ref} on a memory batch, and projects g when the two conflict:

$$\tilde{g} = g - \frac{g^\top g_{\text{ref}}}{g_{\text{ref}}^\top g_{\text{ref}}} g_{\text{ref}} \quad \text{if } g^\top g_{\text{ref}} < 0. \tag{3}$$

Online EWC penalizes movement away from the previous parameters θ^*, weighted by a running Fisher information estimate F decayed by γ:

$$\mathscr{L}_{\text{EWC}} = \ell\big(f_\theta(x), y\big) + \frac{\lambda}{2} \sum_i F_i \big(\theta_i - \theta_i^*\big)^2. \tag{4}$$

SI uses the same quadratic form, with importance weights accumulated from each parameter's contribution to the loss decrease along the training trajectory, scaled by c. LwF was not evaluated because Mammoth implements it only for class- and task-incremental settings.

3.4 Reference models

Joint-Train trains one model on all six training sets at once. It is an empirical offline reference (an approximate ceiling for a single model), not a mathematical bound. **Multi-Model** trains a separate model per category and evaluates each on its own category; because it needs six models and knowledge of the task at test time, it is a reference rather than a competitor.

3.5 Implementation and attribution

Training loops, CL methods and the backbone are taken from Mammoth [1] (pinned commit `e75a491`). Written for this project are: the BMAD dataset plugin for Mammoth (task streaming, three dataset modes, per-category metric logging, prediction and group saving), the data-indexing, grouping and re-splitting module, the split and shortcut audits, the experiment orchestration and tuning protocol, and the statistics and reporting code, covered by 89 unit tests.

4 Experiments

4.1 Data

All six categories come from BMAD [3], resized once to 224×224 RGB. Table 1 summarizes the final balanced splits. The split audit found zero group overlap and zero byte-identical files between any two splits

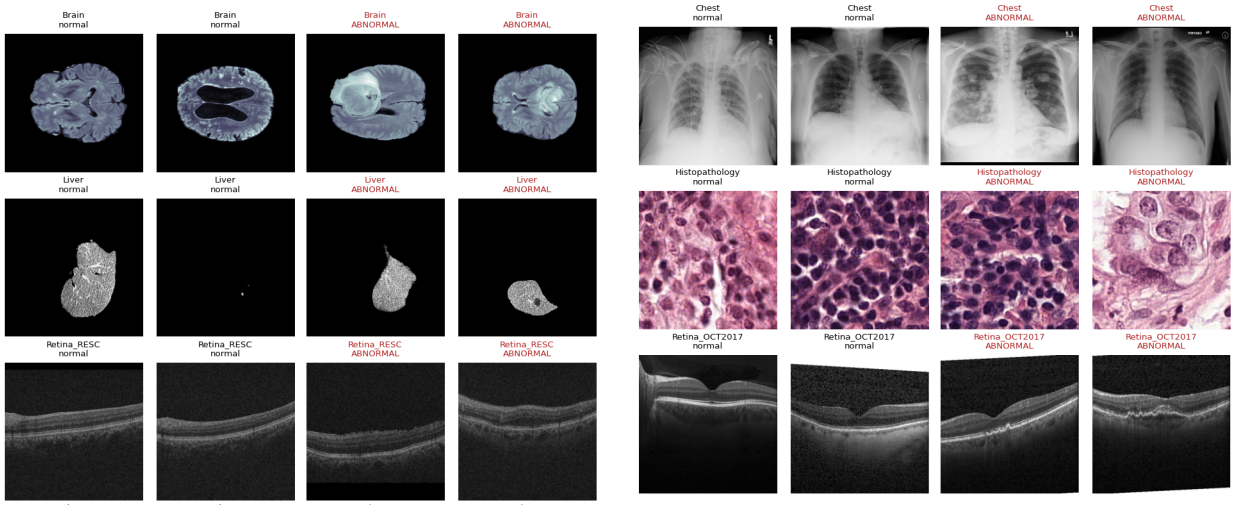


Figure 1: Random training examples from the final splits: two normal (black labels) and two abnormal (red labels) images per category. Left block: Brain, Liver, Retina (RESC); right block: Chest, Histopathology, Retina (OCT2017).

in every category. The input to the model is a single image; the output is the probability that it is abnormal (tumor, lesion, edema, pneumonia, metastasis or retinal pathology, depending on the category). Figure 1 shows randomly drawn training examples.

4.2 Evaluation

Let $R_{i,j}$ denote the test metric on task j after training on task i, for $j \le i$. AUROC is the primary metric because it is threshold-independent; accuracy and F1 at threshold 0.5 are also reported. From R we compute the final average (FAA), the stream average (AvgACC), backward transfer (BWT) and forgetting [4]:

$$\text{FAA} = \frac{1}{T}\sum_{j=1}^{T} R_{T,j}, \qquad \text{AvgACC} = \frac{1}{T}\sum_{i=1}^{T}\frac{1}{i}\sum_{j=1}^{i} R_{i,j}, \tag{5}$$

$$\text{BWT} = \frac{1}{T-1}\sum_{j=1}^{T-1}\left(R_{T,j} - R_{j,j}\right), \qquad \text{Forgetting} = \frac{1}{T-1}\sum_{t=1}^{T-1}\left(\max_{t\le k\le T-1} R_{k,t} - R_{T,t}\right). \tag{6}$$

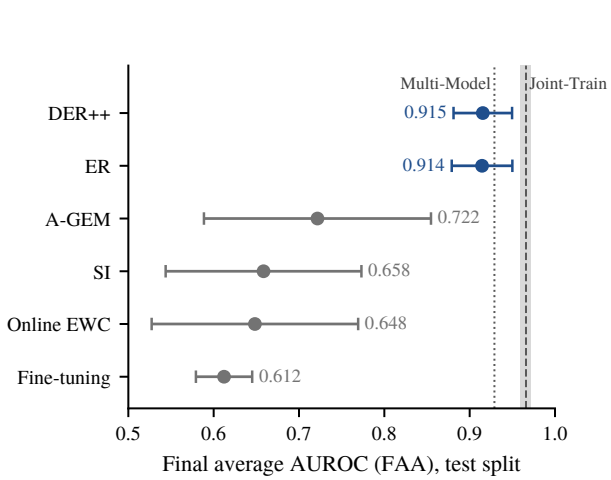
Uncertainty is quantified in three ways: 95% t-intervals across the three training seeds; paired same-seed differences between methods with t-intervals; and bootstrap intervals (2,000 resamples) over the test set of individual runs. The bootstrap resamples whole groups (patients, volumes, blocks) rather than images, because images within a group are correlated; image-level resampling would overstate precision, especially for Liver's 10 volumes.

4.3 Experimental details

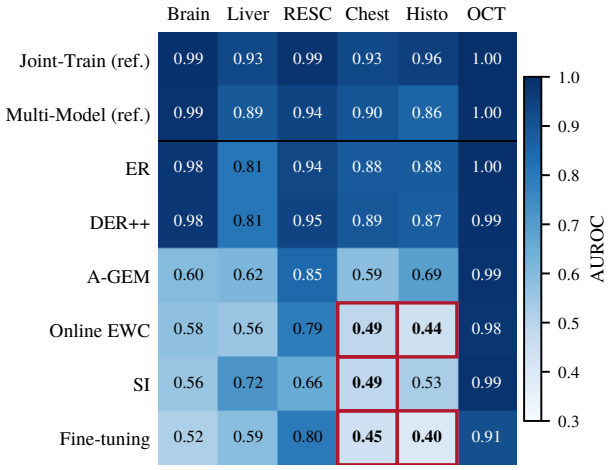
The backbone is a ResNet-18 [11] pretrained on ImageNet, with ImageNet normalization and random horizontal flips. All CL methods use SGD with learning rate 0.01, momentum 0.9, batch size 32 and 5 epochs per task. ER, DER++ and A-GEM share a reservoir memory of 200 images in total. Each method's own hyperparameters were tuned on the validation split with seed 0 (Appendix A): DER++ $\alpha \in \{0.1, 0.3\}$, $\beta \in \{0.5, 1.0\}$; online EWC $\lambda \in \{10, 100, 1000\}$ with $\gamma = 1$; SI $c \in \{0.1, 0.5, 1.0\}$. The selected values were $\alpha = 0.1$, $\beta = 0.5$, $\lambda = 10$ and $c = 1.0$; the $\lambda = 1000$ run diverged during tuning (Section 5.4). Joint-Train was tuned the same way over learning rates $\{0.01, 0.003, 0.001\}$ with constant or step-decayed schedules

Table 2: Test results, mean over 3 seeds ± half-width of the 95% t-interval. AUROC-based scores except the last two columns. Higher is better except for Forgetting. Best CL method per column in bold.

	AUROC				Threshold 0.5	
Method	FAA ↑	AvgACC ↑	BWT ↑	Forgetting ↓	Acc. FAA ↑	F1 FAA ↑
Joint-Train (tuned, reference)	0.966 ±0.006	–	–	–	–	–
Multi-Model (reference)	0.929 ±0.074	–	–	–	–	–
ER	0.914 ±0.036	**0.934** ±0.009	−0.032 ±0.048	0.038 ±0.060	**0.856**	**0.850**
DER++	**0.915** ±0.034	0.915 ±0.034	**−0.008** ±0.016	**0.018** ±0.027	0.838	0.817
A-GEM	0.722 ±0.133	0.752 ±0.052	−0.192 ±0.178	0.196 ±0.169	0.651	0.626
Online EWC (λ=10)	0.648 ±0.121	0.700 ±0.025	−0.281 ±0.064	0.281 ±0.064	0.589	0.270
SI	0.658 ±0.115	0.718 ±0.061	−0.291 ±0.082	0.293 ±0.083	0.588	0.575
Fine-tuning (lower bound)	0.612 ±0.033	0.698 ±0.035	−0.342 ±0.042	0.342 ±0.042	0.560	0.419



(a) Final average AUROC with 95% intervals. Dashed: Joint-Train (shaded: its interval); dotted: Multi-Model.



(b) Final AUROC per category (mean over seeds). Red outline: below 0.5. References above the line.

Figure 2: Replay methods end close to the references on every category; the other methods lose the earliest and most distinct tasks.

(×0.1 after epochs 3 and 4); the selected setting was 0.003 with decay. Final runs use the test split, seeds 0, 1 and 2, and one fixed data split. Each run takes 1–11 minutes on an NVIDIA T4 GPU (Python 3.13, PyTorch 2.11, CUDA 13.0); every run records its environment, command, code-bundle checksum and split-manifest checksum.

4.4 Results

Table 2 and Figure 2a show the main result: replay methods end close to the offline joint-training reference, while every other method ends close to fine-tuning. In a paired same-seed comparison, DER++ minus fine-tuning is $+0.303$ FAA ($[0.239, 0.367]$), so the gain is consistent across seeds. ER minus DER++ is -0.001 ($[-0.056, 0.055]$): the two replay methods are indistinguishable. ER and DER++ trail the tuned Joint-Train model by about 0.05 FAA, and they match the Multi-Model reference (0.929), which uses six separate models. Group-level bootstrap intervals for seed 0 are $[0.905, 0.934]$ for ER, $[0.599, 0.646]$ for fine-tuning and $[0.951, 0.971]$ for Joint-Train, so test-set sampling uncertainty is small relative to the differences between families.

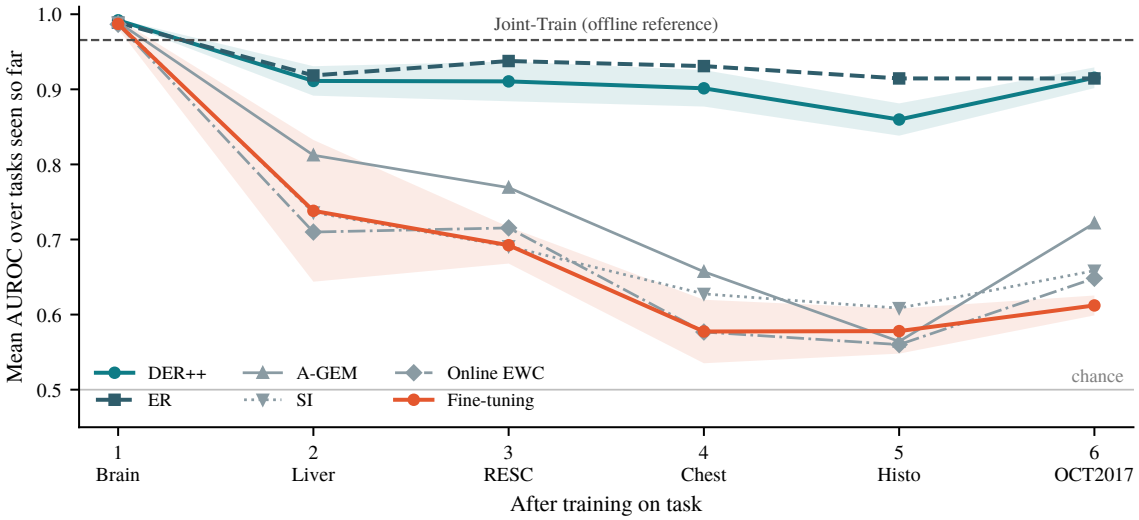


Figure 3: Mean test AUROC over the tasks seen so far, after each task (mean over 3 seeds; shaded bands: $\pm$ one standard deviation for DER++ and fine-tuning).

Because Camelyon16 slide grouping is only approximate (Section 5.5), we repeated the analysis excluding Histopathology from the final average. The conclusions are unchanged: DER++ reaches 0.924 ± 0.043, ER 0.922 ± 0.042 and fine-tuning 0.655 ± 0.108; DER++ minus fine-tuning is $+0.269$ ($[0.123, 0.415]$) and ER minus DER++ is -0.002 ($[-0.060, 0.056]$).

That replay performs best was expected; the size of its margin was not. A memory of 200 images, 2.8% of the 7,244 training images, recovers most of joint training. That regularization performs no better than fine-tuning, even after tuning, was worse than expected. A-GEM, which also uses a memory, ends only 0.11 above fine-tuning, suggesting that using the memory only to constrain gradients, rather than to train on it, is insufficient under strong domain shift.

5 Analysis

5.1 Where forgetting happens

Figure 3 shows how forgetting unfolds over the stream. Replay methods stay within a few points of the offline reference after every task, whereas fine-tuning, the regularization methods and A-GEM fall steeply from task 2 onward and rebound only slightly at the final task, which every method learns easily. Forgetting is concentrated in the earliest and the most visually distinct tasks (Figure 2b).

Under fine-tuning, Brain falls from near-perfect to 0.521 AUROC, essentially chance. Retina (RESC) ends comparatively well (0.800), and the matrices show why (Figure 4): under fine-tuning with seed 0, RESC falls to 0.29 after the Chest task and recovers to 0.88 after OCT2017, the final task and the same imaging modality. Training on a related task can partly restore an earlier one. OCT2017 scores 0.91–1.00 for every method largely because it is the final task, which no later task can overwrite. FAA therefore always includes one task without any opportunity to forget, which slightly flatters weak methods.

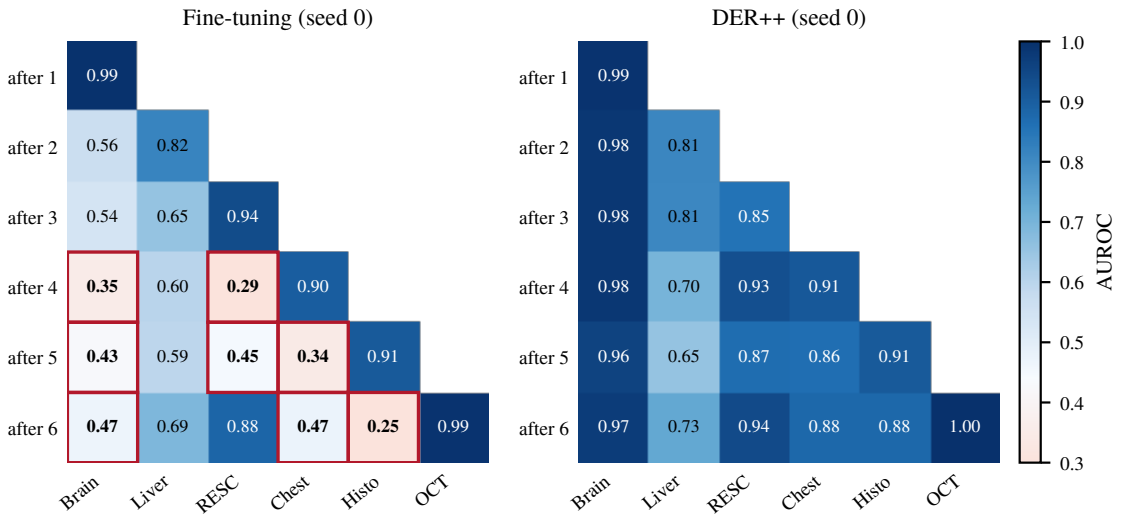


Figure 4: Test AUROC matrices for seed 0: row i shows performance on each task after training on task i. Red outlines mark AUROC below 0.5 (inverted predictions).

5.2 Forgetting can invert the decision rule

Final AUROC falls below 0.5 in five cases: fine-tuning on Histopathology (0.399) and Chest (0.447), online EWC on Histopathology (0.444) and Chest (0.489), and SI on Chest (0.488). An AUROC of 0.40 means that the model ranks normal images above abnormal ones more often than not: rather than merely forgetting, it applies a rule that points the wrong way. In the domain-incremental setting all tasks share one output layer, so a decision direction learned on a later modality can be anti-aligned with an earlier one. Accuracy-based metrics that saturate at 50% conceal this; AUROC exposes it.

5.3 Task order

Table 3: Effect of task order on test FAA (seed 0).

Task order	Fine-tuning	DER++
Barusco et al. [4]: Brain, Liver, RESC, Chest, Histo, OCT2017	0.626	0.900
Reversed: OCT2017, Histo, Chest, RESC, Liver, Brain	0.611	0.922
Mixed: Chest, Brain, OCT2017, Liver, Histo, RESC	0.544	0.924
Range across orders	0.082	0.024

Task order changes fine-tuning substantially more than replay (Table 3). This supports the concern of Qazi et al. [20] that single-order results can mislead, and indicates that replay results are the more order-robust. These are single-seed runs, and the run-to-run variability reported in Section 5.4 is of comparable size to the replay range, so multi-seed order studies are needed before drawing finer conclusions.

5.4 Online EWC: instability and run-to-run variability

At strong regularization ($\lambda = 1000$), online EWC repeatedly produced a NaN loss, which halts Mammoth's training loop: in this study's final protocol it diverged during validation tuning, and it had diverged in three

test runs during development on earlier versions of the splits. Validation therefore selected $\lambda = 10$, at which EWC trains stably but forgets nearly as much as fine-tuning (forgetting 0.281 versus 0.342). We report this as a property of the method on this stream rather than retuning the shared learning rate for EWC alone, which would break the fairness of the comparison.

A planned sensitivity run with $\lambda = 10$ turned out to repeat exactly the selected configuration, which makes it a useful replication: rerunning the same configuration with the same seeds changed FAA by 0.025, 0.058 and 0.071 for seeds 0, 1 and 2 (mean FAA 0.648 versus 0.636). The most likely cause is non-deterministic GPU computation, since seeds were fixed; a run with deterministic algorithms enabled would confirm it. The effect is of the same order as some of the smaller differences in this report, which is why only differences supported by paired intervals are interpreted.

5.5 Data properties that shape the results

Table 4: Shortcut audit: group-cross-validated AUROC of a logistic regression on nine image statistics only (intensity percentiles, contrast, background share, mean color), compared with the trained models' final AUROC.

	Brain	Liver	RESC	Chest	Histo	OCT2017
Image statistics only	0.818	0.617	0.753	0.638	0.689	0.646
ER (final, mean of 3 seeds)	0.980	0.811	0.936	0.883	0.879	0.999
Multi-Model reference	0.990	0.888	0.945	0.897	0.858	0.999

- **Shortcut audit.** Simple image statistics predict the label only moderately (Table 4), well below the trained models in every category, so the models use far more than global intensity or background cues. Brain is highest (0.818) and has two plausible explanations that this audit cannot separate. It may be genuine signal, since tumors appear as bright regions on MRI and shift intensity percentiles. It may also be a slice-position confound: every BraTS subject has a tumor, so normal brain images are, as we understand BMAD's construction, tumor-free slices of the same subjects, which may come disproportionately from the top or bottom of the brain and differ in anatomy rather than pathology. Testing this requires slice indices or a position-matched comparison. The audit is a screen for simple confounds, not proof of their absence; perceptual near-duplicate detection and source-stratified evaluation are not yet implemented.
- **Filename conventions break naive grouping.** Grouping by a filename prefix silently failed on two categories in earlier versions of this pipeline: Chest files named `Lung_Opacity_<id>` all fell into one "patient", and OCT2017's hyphenated patient identifiers were missed, making every image its own group. Both were found by inspecting the real filenames and are fixed; the independent split audit now guards against such failures.
- **Source matching matters.** In an intermediate version, about 90% of Camelyon16 training normals came from tumor-free training slides, while test normals came from the healthy regions of tumor slides; the single-category Histopathology model then fell to 0.48 AUROC on one seed. Drawing training normals preferentially from the same source population as the test normals, with groups still disjoint, removed the mismatch (Section 3.2). A residual seed effect remains: the final single-category Histopathology runs score 0.697, 0.945 and 0.930.
- **Liver is data-scarce and mixes sources in BMAD.** It contains only 10 CT volumes and 218 training images per class, yet improves under joint training (0.931) relative to a dedicated model (0.888), evidence of positive transfer. BMAD's liver training normals come from BTCV [15] while all labeled images come from LiTS, so liver normals are drawn from LiTS only; a tissue-fraction check found no class difference either way (0.064 normal versus 0.063 abnormal).

- **Camelyon16 grouping is approximate.** Its patch files carry no slide identifier, so blocks of 25 consecutive patch numbers approximate slides; patches of one slide may occasionally span two blocks.

5.6 Reference models require tuning too

With the shared constant learning rate, an earlier untuned Joint-Train model was unstable across seeds (accuracy on Brain of 94.9% for one seed and 52.9% for another), and with that reference replay appeared to outperform joint training, which would have been a misleading conclusion. Tuning its learning rate and schedule on validation yields test FAA of 0.963, 0.968 and 0.966 across seeds. Reference models deserve the same tuning effort as the methods compared against them.

6 Conclusion

On a six-modality domain-incremental stream built from BMAD, a 200-image replay memory recovers most of what joint training achieves: DER++ and ER reach 0.915 and 0.914 final AUROC against 0.966 for joint training, match a reference of six separate models, and forget roughly a tenth as much as fine-tuning. Online EWC and SI end within 0.05 of fine-tuning (0.612) and A-GEM 0.11 above it, despite validation tuning. Forgetting can invert decision rules, task order moves fine-tuning by 0.08 AUROC but replay by 0.02, and online EWC is unstable at strong regularization. Beyond these results, the work contributes a reusable, tested pipeline: a supervised BMAD stream with audited patient-, volume- and block-level splits, a protocol that keeps the test set out of all tuning, and reporting that includes intervals, paired comparisons, group-level bootstrap and failed runs.

Limitations. The study uses one fixed data split and three training seeds, and run-to-run GPU variability (up to 0.07 FAA) is comparable to small effects; task-order results are single-seed. The tuning budget was small: shared hyperparameters (learning rate, epochs, memory size) were fixed for the CL methods, and LwF could not be evaluated in Mammoth's domain-incremental setting. Camelyon16 slide grouping is approximate, Liver has only 10 volumes and RESC 15, so their per-category estimates are uncertain. The stream is simulated from separate public datasets rather than drawn chronologically from one institution, and results are not comparable with BMAD's or PatchCoreCL's anomaly-detection numbers, because the task and splits differ.

Future work. Natural next steps are a continual anomaly-detection track on the same stream (PatchCoreCL in PyCLAD), and memory-constrained or privacy-aware settings (smaller memories, or generative and feature-level replay instead of stored patient images). Several audits identified in this work also remain open: (i) a diagnostic study of online EWC instability, logging gradient, parameter and penalty norms and Fisher statistics, over a finer λ grid with lower learning rates and gradient clipping, reported separately from the fair comparison; (ii) a repeat with deterministic algorithms enabled, to confirm the source of run-to-run variability; (iii) perceptual near-duplicate detection and source-stratified evaluation; (iv) recovering true Camelyon16 slide identifiers, and a slice-position-matched test of the Brain shortcut result; and (v) multi-seed task-order studies.

Acknowledgements

I thank Prof. Roberto Corizzo for proposing the scenario and pointing to the relevant literature and tools. Code development used an AI coding assistant (Anthropic's Claude); the experimental design, runs and interpretation were reviewed by the author.

References

[1] AImageLab. Mammoth: An extendible (general) continual learning framework for PyTorch. https://github.com/aimagelab/mammoth, 2026. Commit e75a491, accessed October 2026.

[2] Ujjwal Baid et al. The RSNA-ASNR-MICCAI BraTS 2021 benchmark on brain tumor segmentation and radiogenomic classification. *arXiv preprint arXiv:2107.02314*, 2021.

[3] Jinan Bao, Hanshi Sun, Hanqiu Deng, Yinsheng He, Zhaoxiang Zhang, and Xingyu Li. BMAD: Benchmarks for medical anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2024. arXiv:2306.11876.

[4] Manuel Barusco et al. Towards continual visual anomaly detection in the medical domain. *arXiv preprint arXiv:2508.18013*, 2025. ACM, doi:10.1145/3746259.3760434.

[5] Patrick Bilic et al. The liver tumor segmentation benchmark (LiTS). *Medical Image Analysis*, 84, 2023.

[6] Matteo Boschini, Lorenzo Bonicelli, Pietro Buzzega, Angelo Porrello, and Simone Calderara. Class-incremental continual learning into the eXtended DER-verse. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

[7] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: a strong, simple baseline. In *Advances in Neural Information Processing Systems*, 2020.

[8] Arslan Chaudhry, Marc'Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with A-GEM. In *International Conference on Learning Representations*, 2019.

[9] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc'Aurelio Ranzato. On tiny episodic memories in continual learning. *arXiv preprint arXiv:1902.10486*, 2019.

[10] Babak Ehteshami Bejnordi et al. Diagnostic assessment of deep learning algorithms for detection of lymph node metastases in women with breast cancer. *JAMA*, 318(22), 2017.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

[12] Junjie Hu, Yuanyuan Chen, and Zhang Yi. Automated segmentation of macular edema in OCT using deep neural networks. *Medical Image Analysis*, 55, 2019.

[13] Daniel S. Kermany et al. Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell*, 172(5), 2018.

[14] James Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 2017.

[15] Bennett Landman et al. MICCAI multi-atlas labeling beyond the cranial vault: Workshop and challenge. In *MICCAI Workshop*, 2015.

[16] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12), 2018.

[17] LifelongLab. pyCLAD: Python library for continual lifelong anomaly detection. https://github.com/lifelonglab/pyCLAD, 2026. Accessed October 2026.

[18] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 2017.

[19] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

[20] Mohammad Areeb Qazi et al. Continual learning in medical imaging: A survey and practical analysis. *ACM Computing Surveys*, 2025. doi:10.1145/3785663; arXiv:2405.13482.

[21] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

[22] Jonathan Schwarz, Wojciech Czarnecki, Jelena Luketina, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. Progress & compress: A scalable framework for continual learning. In *International Conference on Machine Learning*, 2018.

[23] George Shih et al. Augmenting the national institutes of health chest radiograph dataset with expert annotations of possible pneumonia. *Radiology: Artificial Intelligence*, 1(1), 2019.

[24] Gido M. van de Ven and Andreas S. Tolias. Three scenarios for continual learning. *arXiv preprint arXiv:1904.07734*, 2019.

[25] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International Conference on Machine Learning*, 2017.

A Hyperparameter Selection

Each method's own hyperparameters were selected by validation FAA (AUROC) with seed 0; the test split was not used for any selection. For Joint-Train (a single task), the score is the mean validation AUROC over the six categories.

Table 5: Validation results of the hyperparameter search. Selected configurations in bold.

Method	Configuration	Val. FAA	Method	Configuration	Val. FAA
DER++	$\alpha = 0.1, \beta = 0.5$	**0.891**	SI	$c = 1.0$	**0.615**
DER++	$\alpha = 0.3, \beta = 0.5$	0.881	SI	$c = 0.5$	0.613
DER++	$\alpha = 0.3, \beta = 1.0$	0.875	SI	$c = 0.1$	0.579
DER++	$\alpha = 0.1, \beta = 1.0$	0.856	Joint-Train	**lr 0.003, step decay**	**0.935**
Online EWC	$\lambda = 10$	**0.699**	Joint-Train	lr 0.01, step decay	0.933
Online EWC	$\lambda = 100$	0.588	Joint-Train	lr 0.001, step decay	0.930
Online EWC	$\lambda = 1000$	diverged	Joint-Train	lr 0.01, constant	0.926
			Joint-Train	lr 0.003, constant	0.921

B Framework Check on a Standard Benchmark

Before building the medical stream, the pipeline was checked on Seq-CIFAR-10 (five tasks, one epoch each, class-incremental). Fine-tuning ended at 17.3% class-incremental accuracy, with the first four tasks at 0%, whereas DER++ with a 500-image memory reached 43.9%, reproducing the qualitative gap expected from the literature.

C Reproducibility

- **Splits:** deterministic given split seed 0; a manifest lists every file's split and group, and its SHA-256 checksum is logged with every result. The manifest is rewritten as each task loads, so the logged checksum identifies the manifest at that stage; all 30 runs in the main task order end on the same final manifest (`55f878bd...`). An independent audit verifies zero group overlap and zero byte-identical files across splits.
- **Verification:** every table in this report was recomputed from the raw per-task metrics log (813 evaluation rows) and matches the pipeline's own output to within floating-point rounding.
- **Code:** the repository is distributed as a checksummed bundle (SHA-256 `7c0a388a...`), unpacked and verified before each session, with 89 unit tests run in the execution environment.
- **Provenance:** every run stores its exact command, Python, PyTorch, CUDA and GPU versions, installed packages, Mammoth commit and code-bundle checksum.
- **Resumability and failures:** completed runs are detected from the metrics log and skipped; runs that crashed are logged and never retried automatically.
- **Variability:** deterministic algorithms were not enforced; an identical configuration rerun with fixed seeds changed FAA by up to 0.07 (Section 5.4), most likely through non-deterministic GPU kernels.